Efficient JPEG Restoration in the Wavelet Domain via Mean Flows

Ştefan-Alexandru Asandei
Independent Researcher
asandei.stefanel@gmail.com

Mihai-Alexandru Radu
Faculty of Physics
"Alexandru Ioan Cuza" University of Iaşi
mihai.radu@student.uaic.ro

***Abstract*—Latest JPEG restoration systems achieve strong quality with large models, yet often remain too slow and expensive for efficient on-device deployment. We present a 65M-parameter generative restorer that attains the lowest LPIPS among the compared settings at QF 10 and 20 on LIVE-1, Urban100, and DIV2K-val while sustaining 8.05 images/s at 1024×1024 on a single RTX 3090. Trained from scratch, the model replaces the learned VAE encoder-decoder with an exactly invertible two-level Haar transform, predicts a clean wavelet-domain residual through a rank-enhanced linear-attention DiT that estimates compression severity internally, and is optimized with an improved MeanFlow objective that enables inference in one or two network evaluations without distillation. Large pretrained priors remain stronger under severe compression (QF 5), whereas our model prioritizes throughput for deployment-constrained restoration.**

I. Introduction

JPEG compression removes high-frequency information and introduces blocking, ringing, and color discontinuities [1]. Feed-forward restorers often over-smooth, whereas DiffBIR, SUPIR, and SODiff reuse large pretrained diffusion priors [2], [3], [4]. Even one-step latent restoration retains the cost of a large denoiser, VAE encoding and decoding, and semantic conditioning.

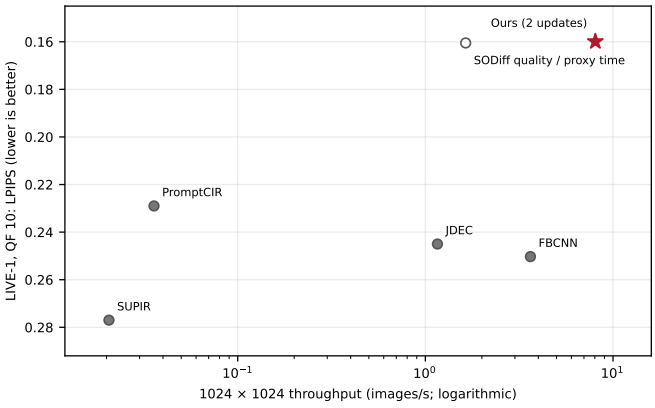


Fig. 1. LIVE-1 QF 10 quality versus 1024^2 throughput. The hollow point pairs published SODiff quality with an SD 2.1 timing proxy, not a measured SODiff system. Other points use local measurements; inference settings remain method-specific.

We train a 65.32M-parameter restorer from scratch without a text encoder, pretrained generator, or VAE. A two-level Haar transform organizes clean-residual prediction [5], [6], and an improved MeanFlow objective [7], [8] enables one- or two-evaluation inference. The resulting quality-efficiency trade-off is strongest at QF 10 and 20; large pretrained priors remain stronger under severe compression and on several perceptual metrics.[1]

Our contributions are:

- a compact, latent-free conditional MeanFlow formulation for JPEG restoration using clean residuals in a two-level Haar domain;
- a SANA-style linear DiT integrating RELA and internal JPEG-severity estimation; and
- a quality-efficiency study across three datasets, backbone resolution scaling, and complete restoration systems.

II. Related Work

FBCNN estimates compression severity and conditions a convolutional restorer on it [9]; JDEC operates on JPEG coefficients [10]; PromptCIR learns degradation prompts [11]. DiffBIR and SUPIR instead reuse pretrained generative priors [2], [3]. SODiff specializes Stable Diffusion 2.1 to one-step JPEG restoration, retaining its UNet and VAE endpoints [4], [12].

Rectified Flow learns an instantaneous transport velocity [13], while MeanFlow learns interval-average velocity for few-update sampling [7], [8]. We combine this objective with the clean-data prediction of JiT and pixel MeanFlow [5], [6]. DiT provides the transformer backbone [14]; SANA makes attention linear in token count [15], and RELA adds a depthwise-convolution branch to recover local feature diversity [16]. We adopt this operator rather than the full LAformer architecture.

III. Method

A. Problem and wavelet representation

Let $y \in [0,1]^{3\times H\times W}$ denote a clean image and $c = \mathcal{J}_q(y)$ its JPEG-compressed observation under an unknown quality factor q. We seek a stochastic mapping $G(c, \epsilon)$ that suppresses compression artifacts while remaining faithful to the content observed in c.

We use an orthonormal two-level Haar wavelet-packet transform $\mathcal{H}^2$ [17], applying the second level to every first-level subband. Tensor shapes follow $3 \times H \times W \to 12 \times$

[1]Training, evaluation, and verification code: https://github.com/stefanasandei/jpeg-restore. Model weights: https://huggingface.co/asandeistefan/jpeg-restore-meanflows.

$H/2 \times W/2 \to 48 \times H/4 \times W/4$. The transform is parameter-free, invertible, and energy-preserving, avoiding a learned autoencoder's reconstruction error and decoder cost.

Writing $h_c = \mathcal{H}^2(c)$ for the encoded condition, we model the scaled residual

$$x = s\big(\mathcal{H}^2(y) - h_c\big), \qquad s = 16 \tag{1}$$

A predicted residual $\hat{x}$ decodes as $\hat{y} = \mathcal{H}^{-2}(h_c + \hat{x}/s)$, preserving the compressed observation on a fixed skip path.

B. Conditional pixel MeanFlow

We adopt the time convention $t = 0$ for data and $t = 1$ for noise. Given $\epsilon \sim \mathcal{N}(0, I)$, the conditional flow path and its velocity are

$$z_t = (1 - t)x + t\epsilon, \qquad v_c = \epsilon - x. \tag{2}$$

MeanFlow learns the average velocity over $[r, t]$,

$$u(z_t, r, t) = \frac{1}{t - r} \int_r^t v(z_\tau, \tau)\, d\tau, \tag{3}$$

which satisfies $v = u + (t - r)\, du/dt$ along the flow trajectory. Sampling traverses each interval through $z_r = z_t - (t - r)u(z_t, r, t)$.

Following the clean-data parameterization of JiT and pixel MeanFlow [5], [6], the network predicts the clean residual endpoint directly and converts it to an average velocity:

$$x_\theta = f_\theta\big([z_t, h_c], t - r\big), \qquad u_\theta = \frac{z_t - x_\theta}{\max(t, t_{\min})}, \tag{4}$$

with $t_{\min} = 0.05$ and interval-length conditioning $t - r$.

Improved MeanFlow [8] forms the corrected instantaneous velocity $V_\theta = u_\theta + (t - r)\ \mathrm{sg}(du_\theta/dt)$ using a JVP with tangent $(u_\theta(z_t, t, t), 0, 1)$ in (z_t, r, t) and fixed condition. Its adaptive velocity loss is

$$d = \mathrm{mean}\left((V_\theta - v_c)^2\right), \qquad \mathcal{L}_{\mathrm{MF}} = (\mathrm{sg}(d) + 0.01)^{-1} d. \tag{5}$$

Here sg stops gradients through the correction; the average-velocity prediction remains differentiable.

Because x_θ decodes directly into an image estimate, we additionally apply LPIPS [18] to $\mathcal{H}^{-2}(h_c + x_\theta/s)$ when $t < 0.8$. The complete objective uses the same adaptive weighting $a(\ell) = \ell/(\mathrm{sg}(\ell) + 0.01)$ for the perceptual loss and adds severity regression:

$$\mathcal{L} = \mathcal{L}_{\mathrm{MF}} + \lambda_p a(\mathcal{L}_{\mathrm{LPIPS}}) + \lambda_q \left|\hat{d}_q - (1 - q/100)\right|, \tag{6}$$

with $\lambda_p = 0.4$ in the final stage and $\lambda_q = 0.1$. Starting from $z_1 = \epsilon$, inference uses one or two uniform intervals, with one network evaluation per interval.

C. Rank-enhanced linear DiT

The backbone follows the SANA linear-DiT layout [15]: 16 transformer blocks with hidden width 576, 18 heads of dimension 32, and an MLP ratio of 2.5. A 2×2 patch embedding in the two-level Haar domain yields a token grid of $H/8 \times W/8$. The noisy state and the degraded condition are concatenated channel-wise, following the direct image-conditioning strategy of Palette [19], and cross-attention is disabled.

For N tokens, vanilla linear attention reassociates the attention product as $\phi(Q)(\phi(K)^T V)$ and therefore scales linearly in N, but the resulting attention map has rank at most the head channel dimension. We insert the RELA branch into every block:

$$\mathrm{RELA}(Q, K, V) = \frac{\phi(Q)(\phi(K)^T V)}{\phi(Q)(\phi(K)^T \mathbf{1})} + \mathrm{DWConv}_{5\times5}(V), \tag{7}$$

where ϕ is ReLU in our implementation. The depthwise convolution contributes a local, potentially full-rank path at small parameter and runtime cost.

The JPEG severity is unknown at inference and is estimated internally. After the first three DiT blocks, global average pooling and a small MLP predict $\hat{d}_q \in [0, 1]$, whose embedding is added to the interval embedding for the remaining 13 blocks. This design is inspired by FBCNN's explicit quality prediction, except that the predicted value modulates the same transformer rather than a separate decoder [9].

a) JVP implementation: We adapt NVIDIA rCM-style segmentation to RELA [20]: `torch.func.jvp` processes embeddings, blocks, severity conditioning, and output separately, detaching outgoing tangents while retaining primal graphs. Both implementations stop gradients through the final derivative correction. Section IV-E compares these existing techniques.

IV. Experiments

A. Datasets, training, and evaluation

a) Training data: We train on DF2K (3,450 images) [21] and all 10,000 images in the first two LSDIR shards (`shard-00` and `shard-01`) [22]. The final checkpoint comprises approximately 101k optimizer updates: 220 epochs on the combined 13,450-image pool with random 128×128 crops at batch size 32, followed by seven epochs on a 5,000-image LSDIR shard with 368×368 crops at batch size 4. The first stage samples the QF uniformly from 5-40, the large-crop continuation concentrates on QF 5-20. Random horizontal and vertical flips provide the only augmentation.

Our restoration model is trained from scratch on a single RTX 3090 with AdamW [23] ($\beta = (0.9, 0.95)$, no weight decay) and gradient-norm clipping at 5. The base learning rate is 5×10^{-5} after warm-up and is reduced to 2×10^{-5} for the large-crop continuation, the LPIPS weight is 0.2 in the base stage and 0.4 in the final stage. Reported MeanFlow training uses $K = 1$.

One final checkpoint is used across all datasets and QFs without dataset-specific tuning.

b) Evaluation protocol: We evaluate LIVE-1 [24], Urban100 [25], and DIV2K-val [21] at QF $\{5, 10, 20\}$. Our model restores full images with two MeanFlow updates and seed zero. OpenCV encodes the RGB references as JPEG, and decoded RGB values are scaled to $[0, 1]$; QF is used only for input

synthesis. Model padding is removed before scoring, without tiled inference, border shaving, or output rounding. LPIPS (AlexNet) [18] and DISTS [26] use TorchMetrics; MUSIQ [27] and MANIQA [28] use `pyiqa`; CLIPIQA [29] uses CLIP ViT-B/16 with fixed prompt pairs. Scores are averaged per image and reported separately for each dataset and QF. Illustrative LSDIR crops are separate from these quantitative benchmarks.

Ours and the five other baselines were evaluated locally; only SODiff quality scores are taken from its paper [4], without evaluating a SODiff checkpoint. Its timing is an SD 2.1 architectural proxy (Section IV-D). The current harness shares JPEG synthesis, scoring, and timing across adapters; JDEC receives coefficients and RGB models decoded inputs. Its full-image PromptCIR path differs from the historical tiled ensemble. The linked repository distinguishes original summaries, executable protocols, and new audit records.

B. Main results

Table I shows competitive LPIPS at QF 10 and 20. At QF 20, LPIPS is lower than SODiff by 25.9% on LIVE-1, 36.4% on Urban100, and 23.2% on DIV2K-val. At QF 10, the respective margins are 0.4%, 6.5%, and 2.1%. Under severe compression, SODiff improves LIVE-1 LPIPS from our 0.2864 to 0.2229. These results indicate a useful operating point, rather than overall restoration superiority.

TABLE I

IMAGE-LEVEL QUALITY AT QF 5, 10, AND 20. BEST AND SECOND-BEST SCORES ARE BOLD AND UNDERLINED. ALL SCORES ARE LOCAL EVALUATIONS EXCEPT SODIFF, QUOTED FROM [4]; DIFFBIR* DENOTES JPEG-SPECIFIC RETRAINING. OURS USES TWO FULL-IMAGE MEANFLOW UPDATES.

Method	LPIPS ↓			DISTS ↓			MUSIQ ↑			MANIQA ↑			CLIPIQA ↑		
	QF5	QF10	QF20	QF5	QF10	QF20	QF5	QF10	QF20	QF5	QF10	QF20	QF5	QF10	QF20
LIVE-1															
JPEG	0.4384	0.3013	0.1799	0.3242	0.2387	0.1653	40.33	53.88	64.12	0.2294	0.3509	0.4411	0.1716	0.2737	0.5542
FBCNN	0.3736	0.2503	0.1583	0.2353	0.1785	0.1319	63.56	71.00	73.96	0.3425	0.4207	0.4551	0.2763	0.4767	0.5535
JDEC	0.4113	0.2450	0.1555	0.2364	0.1740	0.1282	55.66	70.80	73.81	0.2002	0.4065	0.4433	0.1539	0.4811	0.5512
PromptCIR	0.3797	0.2290	0.1450	0.2334	0.1658	0.1223	60.34	72.39	**74.12**	0.2790	0.4500	0.4713	0.2655	0.5176	0.5847
DiffBIR*	0.3509	0.2160	0.1500	0.2035	0.1319	0.0988	58.09	67.38	71.08	0.2812	0.3789	0.4371	0.3776	0.5789	0.6814
SUPIR	0.4856	0.2770	0.1683	0.2720	0.1558	0.1121	52.69	68.77	73.02	0.3229	0.5183	**0.6237**	0.3149	0.6115	0.7364
SODiff	**0.2229**	0.1605	0.1237	**0.1173**	**0.0938**	**0.0763**	**72.88**	**73.84**	74.11	**0.4957**	**0.5192**	0.5272	**0.7087**	**0.7323**	**0.7587**
Ours	0.2864	**0.1599**	**0.0917**	0.2070	0.1443	0.1008	63.37	68.99	71.90	0.2602	0.3325	0.3751	0.6083	0.6846	0.7358
Urban100															
JPEG	0.3481	0.2254	0.1244	0.2834	0.2145	0.1521	50.46	60.87	67.60	0.3656	0.4401	0.4967	0.2806	0.3517	0.5343
FBCNN	0.2341	0.1462	0.0896	0.2162	0.1648	0.1249	69.03	72.55	73.39	0.4263	0.5033	0.5288	0.3800	0.5014	0.5437
JDEC	0.2794	0.1382	0.0846	0.2309	0.1570	0.1175	62.97	72.52	73.30	0.3386	0.5001	0.5230	0.2518	0.4959	0.5369
PromptCIR	0.2389	0.1183	0.0739	0.2037	0.1431	0.1083	66.08	**73.01**	**73.47**	0.3946	0.5380	0.5489	0.3619	0.5337	0.5662
DiffBIR*	0.2018	0.1344	0.1005	0.1657	0.1207	0.0939	69.63	71.77	72.51	0.4285	0.4813	0.5105	0.5470	0.5966	0.6306
SUPIR	0.3279	0.2489	0.2125	0.2018	0.1659	0.1518	69.94	72.37	73.01	0.5546	**0.5995**	**0.6105**	0.5536	0.6178	0.6397
SODiff	**0.1579**	0.1098	0.0846	**0.1196**	**0.0914**	**0.0734**	71.33	72.23	72.63	**0.5598**	0.5451	0.5561	0.6392	0.6558	0.6733
Ours	0.2045	**0.1027**	**0.0538**	0.2560	0.1880	0.1368	65.31	68.92	70.24	0.3598	0.4258	0.4682	**0.6428**	**0.6939**	**0.7172**
DIV2K-val															
JPEG	0.3459	0.3234	0.2072	0.2570	0.2255	0.1465	25.95	47.53	57.45	0.2570	0.3120	0.3557	0.2595	0.3303	0.5072
FBCNN	0.3445	0.2448	0.1733	0.2078	0.1581	0.1168	56.52	61.79	65.20	0.3025	0.3593	0.3775	0.3004	0.4561	0.5221
JDEC	0.3811	0.2313	0.1565	0.2234	0.1574	0.1152	53.88	**67.48**	**69.90**	0.2118	0.3689	0.3927	0.1841	0.4675	0.5319
PromptCIR	0.3549	0.2240	0.1581	0.2067	0.1459	0.1061	52.21	62.63	65.62	0.2705	0.3758	0.3871	0.3041	0.4956	0.5483
DiffBIR*	0.2788	0.1953	0.1542	0.1533	0.1072	0.0856	60.21	65.22	67.06	0.3220	0.3754	**0.4033**	0.4975	0.5912	0.6355
SUPIR	0.4372	0.3121	0.2295	0.2148	0.1410	0.1161	54.07	61.93	64.87	0.3438	0.3570	0.3723	0.4219	0.5186	0.5535
SODiff	**0.2425**	0.1732	0.1295	**0.1126**	**0.0816**	**0.0622**	**64.42**	65.90	66.49	**0.3733**	**0.3924**	0.3984	0.5851	0.6193	0.6398
Ours	0.2821	**0.1695**	**0.0995**	0.1800	0.1209	0.0828	57.36	62.30	64.62	0.2494	0.3103	0.3427	**0.6048**	**0.6558**	**0.6820**

SODiff is strongest on DISTS, and large pretrained priors generally lead on MUSIQ and MANIQA. Our CLIPIQA is strongest on Urban100 and DIV2K-val at every tested QF. The metric-dependent ordering does not establish overall superiority.

C. Resolution scaling

An $H \times W$ image yields $N = HW/64$ tokens after the Haar and patch transforms. Doubling each dimension increases full-attention cost by $16\times$, whereas linear attention scales linearly in N but constrains the attention-map rank. RELA adds a local depthwise-convolution path while retaining linear token scaling [15], [16].

Figure 2 and Table II compare one forward pass through four capacity-matched architectures, using the two-level Haar representation, batch size one, BF16, PyTorch eager execution, and one RTX 3090. Inputs are square. These measurements isolate backbone cost rather than two-update end-to-end restoration. At 1440^2, RELA is $2.9\times$ faster than full-attention DiT and incurs a 5.7% throughput cost relative to SANA. Restormer [30] uses 2.79 GiB versus RELA's 0.81 GiB. Full-attention DiT uses less forward-only memory than RELA but pays for quadratic attention in latency; below 512^2, fixed costs dominate and it is fastest.

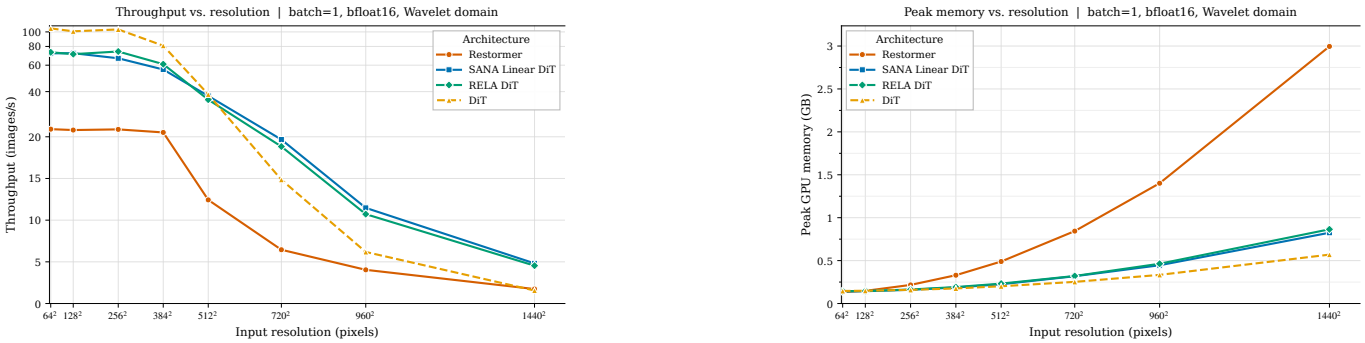


Fig. 2. One-forward-pass scaling in the two-level Haar domain, batch size 1, BF16, RTX 3090. Left: throughput. Right: peak allocated memory. These measurements isolate backbone cost.

TABLE II
EAGER ONE-FORWARD-PASS ARCHITECTURE BENCHMARK: BATCH SIZE 1, BF16, TWO-LEVEL HAAR, RTX 3090. RESOLUTIONS ARE SQUARE.

Resolution	Restormer (57.12M) ms	img/s	GiB	SANA Linear DiT (64.34M) ms	img/s	GiB	RELA DiT (65.32M) ms	img/s	GiB	DiT (67.47M) ms	img/s	GiB
64^2	44.4707	22.4867	0.12538	13.8961	71.9626	0.13189	13.6673	73.1673	0.13368	9.4790	105.496	0.13638
128^2	45.0920	22.1769	0.13728	13.8650	72.1242	0.13371	14.0712	71.0673	0.13571	9.9097	100.911	0.13871
256^2	44.6283	22.4073	0.20189	14.9908	66.7077	0.14912	13.5093	74.0231	0.15194	9.6308	103.834	0.14805
384^2	46.7692	21.3816	0.30720	17.8032	56.1697	0.17399	16.4050	60.9571	0.17860	12.356	80.9325	0.16362
512^2	80.4425	12.4312	0.45579	26.7253	37.4178	0.20924	28.2868	35.3522	0.21598	26.0815	38.3414	0.18736
720^2	155.055	6.4493	0.78590	50.8455	19.6674	0.29607	53.1076	18.8297	0.29880	67.3489	14.8480	0.23488
960^2	247.412	4.0418	1.3052	87.1724	11.4715	0.41543	93.2710	10.7214	0.43167	161.674	6.1853	0.31070
1440^2	575.833	1.7366	2.7898	207.718	4.8142	0.76812	220.201	4.5413	0.80525	640.170	1.5621	0.53055

D. System efficiency

Table III reports 8.05 images/s (0.1242 s) for ours, 2.2× the FBCNN throughput. The SD 2.1 proxy does not establish SODiff deployment speed.

TABLE III
REPORTED RTX 3090 BATCH-ONE 1024^2 EFFICIENCY. SD 2.1 PROXY† RECONSTRUCTS THE SODIFF ARCHITECTURE WITHOUT ITS WEIGHTS; OTHER ROWS TIME RESTORATION SYSTEMS. NFE INCLUDES TILES AND ENSEMBLES.

Method	Parameters (M)	NFE	Latency (s)	Throughput (img/s)
FBCNN	70.10	1	0.275	3.63
JDEC	38.90	1	0.860	1.16
PromptCIR	34.75	64	27.97	0.0358
SUPIR	4490	50	48.66	0.0206
SD 2.1 proxy†	1288	1	0.610	1.64
Ours	65.32	2	0.1242	8.05

Original RTX 3090 timings are reported aggregates; raw repetitions and the SD 2.1 proxy script are not archived. The 0.610 s proxy follows SODiff's architecture without its weights and does not establish SODiff latency. The released harness uses five warm-ups and 20 synchronized calls, including endpoints/sampling and excluding loading, metrics, and output. Its protocol cannot retroactively define original timing boundaries.

NFE includes tiles/ensembles. Ours uses two full-image calls; FBCNN/JDEC use one. PromptCIR [11] uses 16 tiles across four checkpoint/transpose passes (64 calls). The Haar/BF16 benchmark separately measures backbone cost.

E. JVP implementation ablation

Table IV audits the trained checkpoint on CPU FP32: batch one, 32^2 synthetic RGB, three noise seeds, identical weights and $(r, t) = (0.2, 0.7)$. Primal outputs, tangents, MeanFlow-plus-severity losses, and all parameter gradients agree; LPIPS is excluded. rRMSE denotes relative L_2 error. This verifies only these CPU cases. Historical GPU memory reductions are not claimed: their accounting is unrecorded. Audit records and CUDA memory definitions are released.

TABLE IV
CPU FP32 JVP AUDIT: TRAINED CHECKPOINT, BATCH ONE, 32^2.

Seed	Tangent rRMSE	$\lvert\Delta\mathcal{L}\rvert$	Gradient rRMSE	Max $\lvert\Delta g\rvert$
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0

V. LIMITATIONS AND FUTURE WORK

Quality uses one seed; no quality ablation isolates Haar, RELA, or severity estimation. Backbone scaling measures cost, not quality attribution. The 13,450-image training pool may

limit semantic knowledge; generated detail need not recover missing information faithfully. Larger-scale training may help. Method-specific precision, tiling, and sampling limit speed comparisons.

VI. Conclusion

Haar-domain residuals and a RELA DiT enable two-update MeanFlow restoration without a VAE or pretrained generator. Our 65.32M-parameter model reaches 8.05 images/s at 1024^2 with competitive LPIPS. Larger pretrained priors retain advantages under severe compression and on other metrics.

Acknowledgments

We thank Radu Miron and Andrei Ciortea for their valuable feedback and helpful discussions.

References

[1] G. K. Wallace, "The JPEG still picture compression standard," *Commun. ACM*, vol. 34, no. 4, pp. 30–44, Apr. 1991, doi: 10.1145/103085.103089.

[2] X. Lin *et al.*, "DiffBIR: Toward blind image restoration with generative diffusion prior," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 430–448, doi: 10.1007/978-3-031-73202-7_25.

[3] F. Yu *et al.*, "Scaling up to excellence: Practicing model scaling for photo-realistic image restoration in the wild," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 25 669–25 680.

[4] T. Yang, J. Gong, J. Guo, W. Li, Y. Guo, and Y. Zhang, "SODiff: Semantic-oriented diffusion model for JPEG compression artifacts removal," 2025, arXiv:2508.07346.

[5] T. Li and K. He, "Back to basics: Let denoising generative models denoise," 2025, arXiv:2511.13720.

[6] Y. Lu *et al.*, "One-step latent-free image generation with pixel mean flows," 2026, arXiv:2601.22158.

[7] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He, "Mean flows for one-step generative modeling," 2025, arXiv:2505.13447.

[8] Z. Geng, Y. Lu, Z. Wu, E. Shechtman, J. Z. Kolter, and K. He, "Improved mean flows: On the challenges of fastforward generative models," 2025, arXiv:2512.02012.

[9] J. Jiang, K. Zhang, and R. Timofte, "Towards flexible blind JPEG artifacts removal," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 4997–5006.

[10] W. K. Han, S. Im, J. Kim, and K. H. Jin, "JDEC: JPEG decoding via enhanced continuous cosine coefficients," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2024, pp. 2784–2793.

[11] B. Li *et al.*, "PromptCIR: Blind compressed image restoration with prompt learning," 2024, arXiv:2404.17433.

[12] Stability AI. "Stable Diffusion 2.1 base". Hugging Face. Accessed: Oct. 3, 2026. [Online]. Available: https://huggingface.co/stabilityai/stable-diffusion-2-1-base

[13] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2023. [Online]. Available: https://openreview.net/forum?id=XVjTT1nw5z

[14] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 4195–4205.

[15] E. Xie *et al.*, "SANA: Efficient high-resolution image synthesis with linear diffusion transformers," 2024, arXiv:2410.10629.

[16] Y. Ai, "Fast and accurate image restoration and generation with rank enhanced linear attention," 2025, arXiv:2505.16157.

[17] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 11, no. 7, pp. 674–693, Jul. 1989, doi: 10.1109/34.192463.

[18] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2018, pp. 586–595.

[19] C. Saharia *et al.*, "Palette: Image-to-image diffusion models," in *Proc. ACM SIGGRAPH Conf.*, Aug. 2022, pp. 1–10, doi: 10.1145/3528233.3530757.

[20] NVIDIA Research, "rCM: Score-regularized continuous-time consistency model," GitHub, 2025, Accessed: Oct. 3, 2026. [Online]. Available: https://github.com/NVlabs/rcm

[21] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jul. 2017, pp. 1122–1131, doi: 10.1109/CVPRW.2017.150.

[22] Y. Li *et al.*, "LSDIR: A large scale dataset for image restoration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2023, pp. 1775–1787.

[23] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[24] H. R. Sheikh, Z. Wang, L. Cormack, and A. C. Bovik, "LIVE image quality assessment database release 2," Laboratory for Image and Video Engineering, Univ. Texas at Austin, 2005, Accessed: Oct. 3, 2026. [Online]. Available: https://live.ece.utexas.edu/research/quality/subjective.htm

[25] J.-B. Huang, A. Singh, and N. Ahuja, "Single image super-resolution from transformed self-exemplars," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 5197–5206, doi: 10.1109/CVPR.2015.7299156.

[26] K. Ding, K. Ma, S. Wang, and E. P. Simoncelli, "Image quality assessment: Unifying structure and texture similarity," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 5, pp. 2567–2581, May 2022, doi: 10.1109/TPAMI.2020.3045810.

[27] J. Ke, Q. Wang, Y. Wang, P. Milanfar, and F. Yang, "MUSIQ: Multi-scale image quality transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5148–5157.

[28] S. Yang *et al.*, "MANIQA: Multi-dimension attention network for no-reference image quality assessment," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2022, pp. 1191–1200.

[29] J. Wang, K. C. K. Chan, and C. C. Loy, "Exploring CLIP for assessing the look and feel of images," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 2, 2023, pp. 2555–2563, doi: 10.1609/aaai.v37i2.25353.

[30] S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, and M.-H. Yang, "Restormer: Efficient transformer for high-resolution image restoration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 5728–5739.

Appendix

Qualitative Results

Illustrative 512×512 LSDIR crops: left, QF 10/two updates; right, QF 5/one update.



QF 10, 2 steps QF 5, 1 step